

executable

(gcc)

cpp/cc1 | ln | ld

```
tailn@mosaic: ~/col700V/com X + v
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
```

For help, type "help".

Type "apropos word" to search for commands related to "word"...

Reading symbols from a.out...

(gdb) list

warning: Source file is more recent than executable.

```
1  #include <stdio.h>
2  int main() {
3      int a,b,c,d;
4      a=34*33;
5      b=a++;
6      c=b+23;
7      d=a+b/c*a;
8      printf("Hello %d\n",d);
9      return 0;
10 }
```

(gdb) b

Very humid
False

Search



ENG
IN 07-10-2025

Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<<https://www.gnu.org/software/gdb/bugs/>>.
Find the GDB manual and other documentation resources online at:
<<http://www.gnu.org/software/gdb/documentation/>>.

For help, type "help".

Type "apropos word" to search for commands related to "word"...

Reading symbols from a.out...

(gdb) list

warning: Source file is more recent than executable.

```
1      #include <stdio.h>
2      int main() {
3          int a,b,c,d;
4          a=34*33;
5          b=a++;
6          c=b+23;
7          d=a+b/c*a;
8          printf("Hello %d\n",d);
9          return 0;
10     }
```

(gdb) b 5

Breakpoint 1 at 0x1165: file showBreak.c, line 5.

(gdb) info breakpoints

Num	Type	Disp	Enb	Address	What
1	breakpoint	keep y		0x00000000000001165	in main at showBreak.c:5

(gdb) di

warning: Source file is more recent than executable.

```

1  #include <stdio.h>
2  int main() {
3      int a,b,c,d;
4      a=34*33;
5      b=a++;
6      c=b+23;
7      d=a+b/c*a;
8      printf("Hello %d\n",d);
9      return 0;
10 }
```

(gdb) b 5

Breakpoint 1 at 0x1165: file showBreak.c, line 5.

(gdb) info breakpoints

Num	Type	Disp	Enb	Address	What
1	breakpoint	keep y		0x0000000000001165	in main at showBreak.c:5

(gdb) disassemble main

Dump of assembler code for function main:

```

0x0000000000001149 <+0>:      endbr64
0x000000000000114d <+4>:      push    %rbp
0x000000000000114e <+5>:      mov     %rsp,%rbp
0x0000000000001151 <+8>:      lea     0xeac(%rip),%rax      # 0x2004
0x0000000000001158 <+15>:     mov     %rax,%rdi
0x000000000000115b <+18>:     call    0x1050 <puts@plt>
0x0000000000001160 <+23>:     mov     $0x0,%eax
0x0000000000001165 <+28>:     pop     %rbp
0x0000000000001166 <+29>:     ret
```

End of assembler dump.

(gdb)

```

6         c=b+23;
7         d=a+b/c*a;
8         printf("Hello %d\n",d);
9         return 0;
10    }

```

(gdb) b 5

Breakpoint 1 at 0x1165: file showBreak.c, line 5.

(gdb) info breakpoints

Num	Type	Disp	Enb	Address	What
1	breakpoint	keep y		0x0000000000001165	in main at showBreak.c:5

(gdb) disassemble main

Dump of assembler code for function main:

```

0x0000000000001149 <+0>:      endbr64
0x000000000000114d <+4>:      push    %rbp
0x000000000000114e <+5>:      mov     %rsp,%rbp
0x0000000000001151 <+8>:      lea     0x00000000(%rip),%rax      # 0x2004
0x0000000000001158 <+15>:     mov     %rax,%rdi
0x000000000000115b <+18>:     call    0x1050 <puts@plt>
0x0000000000001160 <+23>:     mov     $0x0,%eax
0x0000000000001165 <+28>:     pop     %rbp
0x0000000000001166 <+29>:     ret

```

End of assembler dump.

(gdb) b *0x000000000000115b

Breakpoint 2 at 0x115b: file showBreak.c, line 3.

(gdb) info breakpoints

Num	Type	Disp	Enb	Address	What
1	breakpoint	keep y		0x0000000000001165	in main at showBreak.c:5
2	breakpoint	keep y		0x000000000000115b	in main at showBreak.c:3

(gdb) run


```
0x00000000000001149 <+0>:      endbr64
0x0000000000000114d <+4>:      push    %rbp
0x0000000000000114e <+5>:      mov     %rsp,%rbp
0x00000000000001151 <+8>:      lea     0xeac(%rip),%rax      # 0x2004
0x00000000000001158 <+15>:     mov     %rax,%rdi
0x0000000000000115b <+18>:     call   0x1050 <puts@plt>
0x00000000000001160 <+23>:     mov     $0x0,%eax
0x00000000000001165 <+28>:     pop     %rbp
0x00000000000001166 <+29>:     ret
```

End of assembler dump.

(gdb) b *0x0000000000000115b

Breakpoint 2 at 0x115b: file showBreak.c, line 3.

(gdb) info breakpoints

Num	Type	Disp	Enb	Address	What
1	breakpoint	keep y		0x00000000000001165	in main at showBreak.c:5
2	breakpoint	keep y		0x0000000000000115b	in main at showBreak.c:3

(gdb) run

Starting program: /home/kolin/col7001/concurrency/a.out

Warning:

Cannot insert breakpoint 2.

Cannot access memory at address 0x115b

(gdb) x/10xb \$rip

0x7ffff7fe3290:	0x48	0x89	0xe7	0xe8	0x98	0x0d	0x00	0x00
0x7ffff7fe3298:	0x49	0x89						

(gdb) x/10xb \$rip-10

0x7ffff7fe3286:	0x0f	0x1f	0x84	0x00	0x00	0x00	0x00	0x00
0x7ffff7fe328e:	0x66	0x90						

(gdb)

Multiprocessor Types

- Loosely coupled multiprocessors
 - No shared global memory address space
 - Multicomputer network
 - Network-based multiprocessors
 - Usually programmed via message passing
 - Explicit calls (send, receive) for communication
- Tightly coupled multiprocessors
 - Shared global memory address space
 - Traditional multiprocessing: symmetric multiprocessing (SMP)
 - Existing multi-core processors, multithreaded processors
 - Programming model similar to uniprocessors (i.e., multitasking uniprocessor) except
 - Operations on shared data require synchronization

Main Design Issues in Tightly-Coupled MP

- Shared memory synchronization
 - How to handle locks, atomic operations
- Cache coherence
 - How to ensure correct operation in the presence of private caches keeping the same memory address cached
- Memory consistency: Ordering of all memory operations
 - What should the programmer expect the hardware to provide?
- Shared resource management
- Communication: Interconnects

Main Programming Issues in Tightly-Coupled MP

- **Load imbalance**
 - How to partition a single task into multiple tasks
- **Synchronization**
 - How to synchronize (efficiently) between tasks
 - How to communicate between tasks
 - Locks, barriers, pipeline stages, condition variables, semaphores, atomic operations, ...
- **Contention**
- **Maximizing parallelism**
- **Ensuring correct operation while optimizing for performance**

Hardware-based Multithreading

- Coarse grained
 - Quantum based
 - Event based (switch-on-event multithreading), e.g., switch on L3 miss
- Fine grained
 - Cycle by cycle
 - Thornton, "CDC 6600: Design of a Computer," 1970.
 - Burton Smith, "A pipelined, shared resource MIMD computer," ICPP 1978.
- Simultaneous
 - Can dispatch instructions from multiple threads at the same time
 - Good for improving execution unit utilization

Parallel Speedup Example

- $a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0$
- Assume given inputs: x and each a_i
- Assume each operation 1 cycle, no communication cost, each op can be executed in a different processor
- How fast is this with a single processor?

$$((a_4x + a_3)x + a_2)x + a_1)x + a_0$$

executable

(gcc)

cpp	cc1	ln	ld
-----	-----	----	----

Parallel Speedup Example

- $a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0$
- Assume given inputs: x and each a_i
- Assume each operation 1 cycle, no communication cost, each op can be executed in a different processor
- How fast is this with a single processor?
 - Assume no pipelining or concurrent execution of instructions
- How fast is this with 3 processors?

Superlinear Speedup

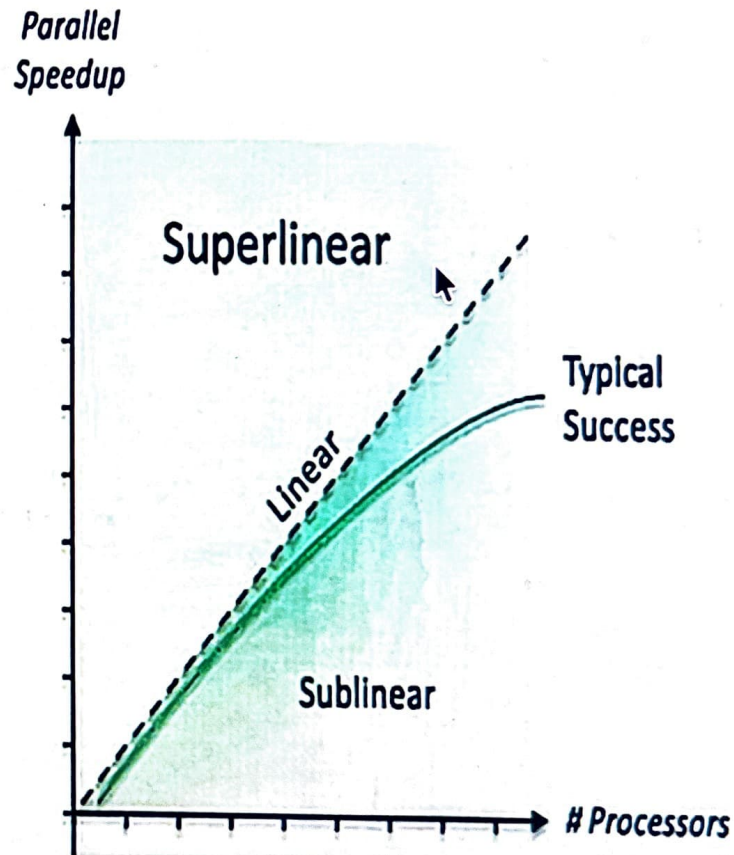
- Can speedup be greater than P with P processing elements?

- Unfair comparisons

Compare best parallel algorithm to wimpy serial algorithm $\rightarrow$ unfair

- Cache/memory effects

More processors $\rightarrow$
more cache or memory $\rightarrow$



Utilization, Redundancy, Efficiency

- Traditional metrics
 - Assume all P processors are tied up for parallel computation
- Utilization: How much processing capability is used
 - $U = (\# \text{ Operations in parallel version}) / (\text{processors} \times \text{Time})$
- Redundancy: how much extra work is done with parallel processing
 - $R = (\# \text{ of operations in parallel version}) / (\# \text{ operations in best single processor algorithm version})$
- Efficiency
 - $E = (\text{Time with 1 processor}) / (\text{processors} \times \text{Time with P processors})$

Caveats of Parallelism (II)

- Amdahl's Law

- f: Parallelizable fraction of a program
- N: Number of processors

$$\text{Speedup} = \frac{1}{1 - f + \frac{f}{N}}$$

- Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," AFIPS 1967.
- Maximum speedup limited by serial portion: Serial bottleneck
- Parallel portion is usually not perfectly parallel
 - Synchronization overhead (e.g., updates to shared data)
 - Load imbalance overhead (imperfect parallelization)

Caveats of Parallelism (II)

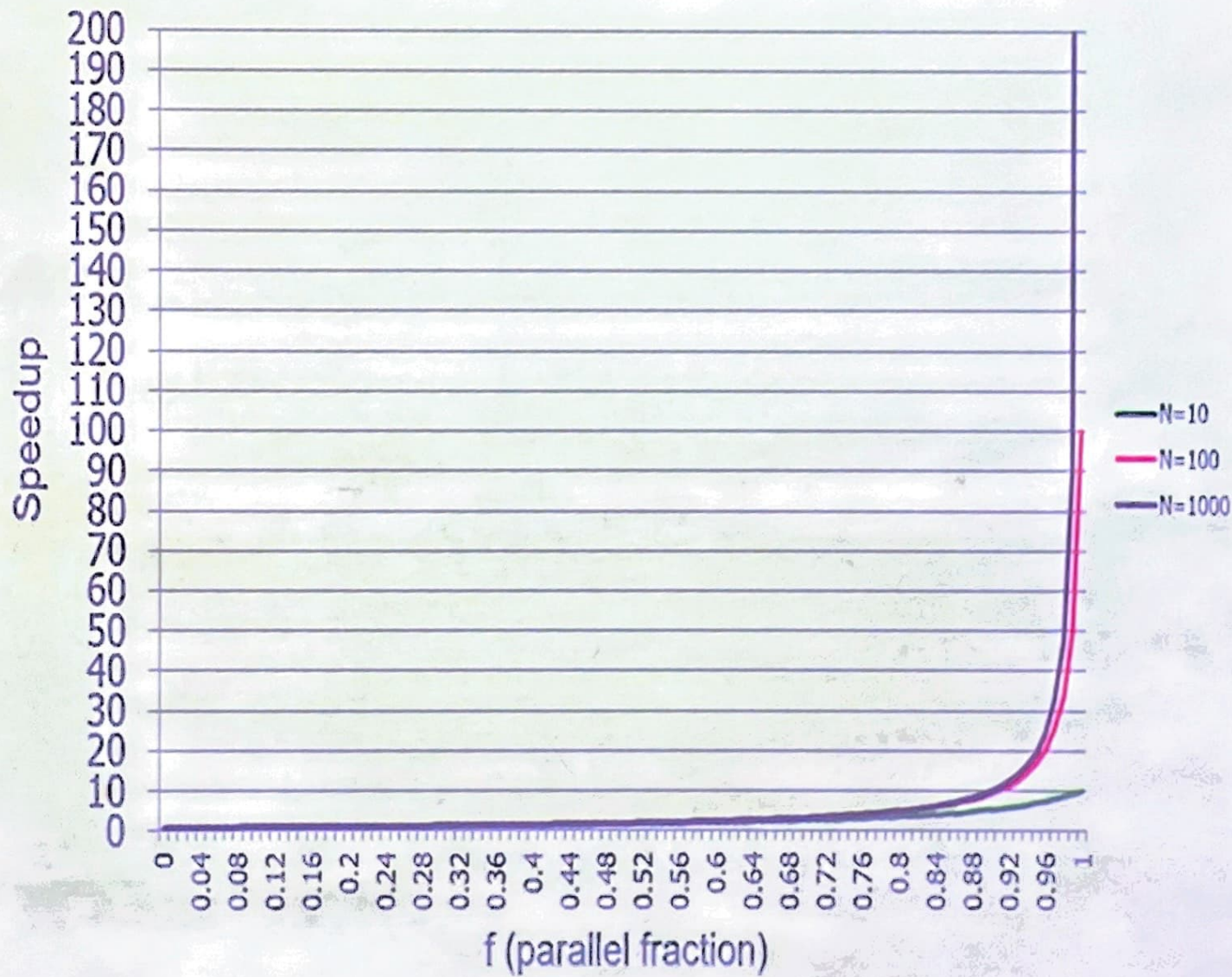
- Amdahl's Law

- f: Parallelizable fraction of a program
- N: Number of processors

$$\text{Speedup} = \frac{1}{1 - f + \frac{f}{N}}$$

- Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," AFIPS 1967.
- Maximum speedup limited by serial portion: Serial bottleneck
- Parallel portion is usually not perfectly parallel
 - Synchronization overhead (e.g., updates to shared data)
 - Load imbalance overhead (imperfect parallelization)
 - Resource sharing overhead (contention among N processors)

Sequential Bottleneck



What is the Maximum Speedup?

- f = fraction of computation (algorithm) that is serial and cannot be parallelized
 - Data setup
 - Reading/writing to a single disk file
- $t_s = ft_s + (1-f) t_s$
= serial portion + parallelizable portion
- $t_p = ft_s + ((1-f) t_s)/n$
- $S(n) = t_s/(ft_s + ((1-f) t_s)/n)$
= $n/(1 + (n-1)f)$ <- Amdahl's Law

Limit as $n \rightarrow \infty = 1/f$

Example of Amdahl's Law

- Suppose that a calculation has a 4% serial portion, what is the limit of speedup on 16 processors?
 - $16 / (1 + (16 - 1) * 0.04) = 10$
 - What is the maximum speedup?
 - $1 / 0.04 = 25$

More to think about ...

- Amdahl's law works on a *fixed* problem size
 - This is reasonable if your only goal is to solve a problem faster.
 - What if you also want to solve a larger problem?
 - Gustafson's Law (Scaled Speedup)

Gustafson's Law

- Fix execution of on a single processor as
 - $s + p = \text{serial part} + \text{parallelizable part} = 1$
- $S(n) = (s + p)/(s + p/n)$
 $= 1/(s + (1 - s)/n) = \text{Amdahl's law}$
- Now let, $1 = s + \pi = \text{execution time on a parallel computer, with } \pi = \text{parallel part.}$
 - $S_s(n) = (s + \pi n)/(s + \pi) = n + (1 - n)s$

More on Gustafson's Law

- Derived by fixing the parallel execution time
 - Amdahl fixed the problem size
 - fixed serial execution time
 - For many practical situations, Gustafson's law makes more sense
 - Have a bigger computer, solve a bigger problem.
- Amdahl's law turns out to be too conservative for high-performance computing.

Efficiency

- $E(n) = S(n)/n * 100\%$
- A program with linear speedup is 100% efficient.

Example questions

- Given a (scaled) speed up of 20 on 32 processors, what is the serial fraction from Amdahl's law?, From Gustafson's Law?
- A program attains 89% efficiency with a serial fraction of 2%. Approximately how many processors are being used according to Amdahl's law?

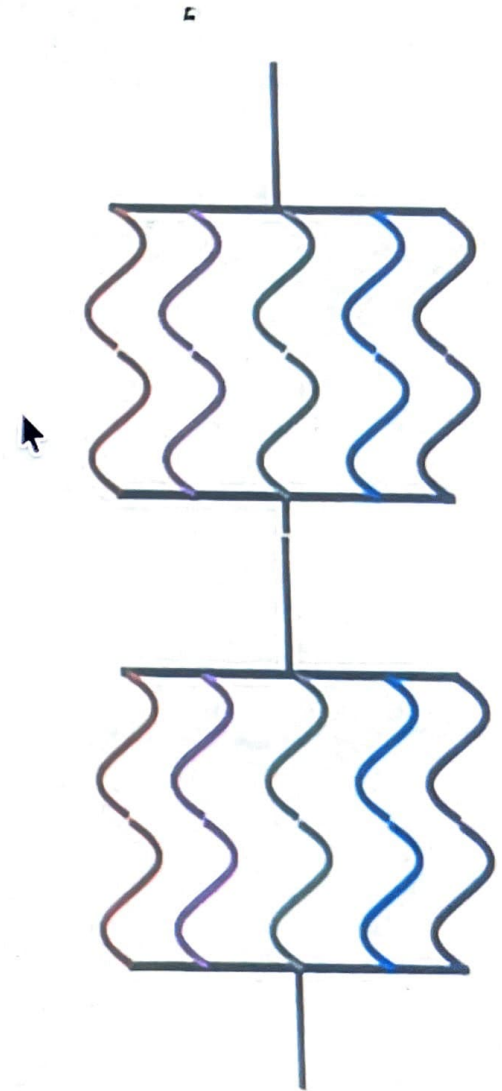
Why the Sequential Bottleneck?

- Parallel machines have the sequential bottleneck
- Main cause: Non-parallelizable operations on data (e.g. non-parallelizable loops)

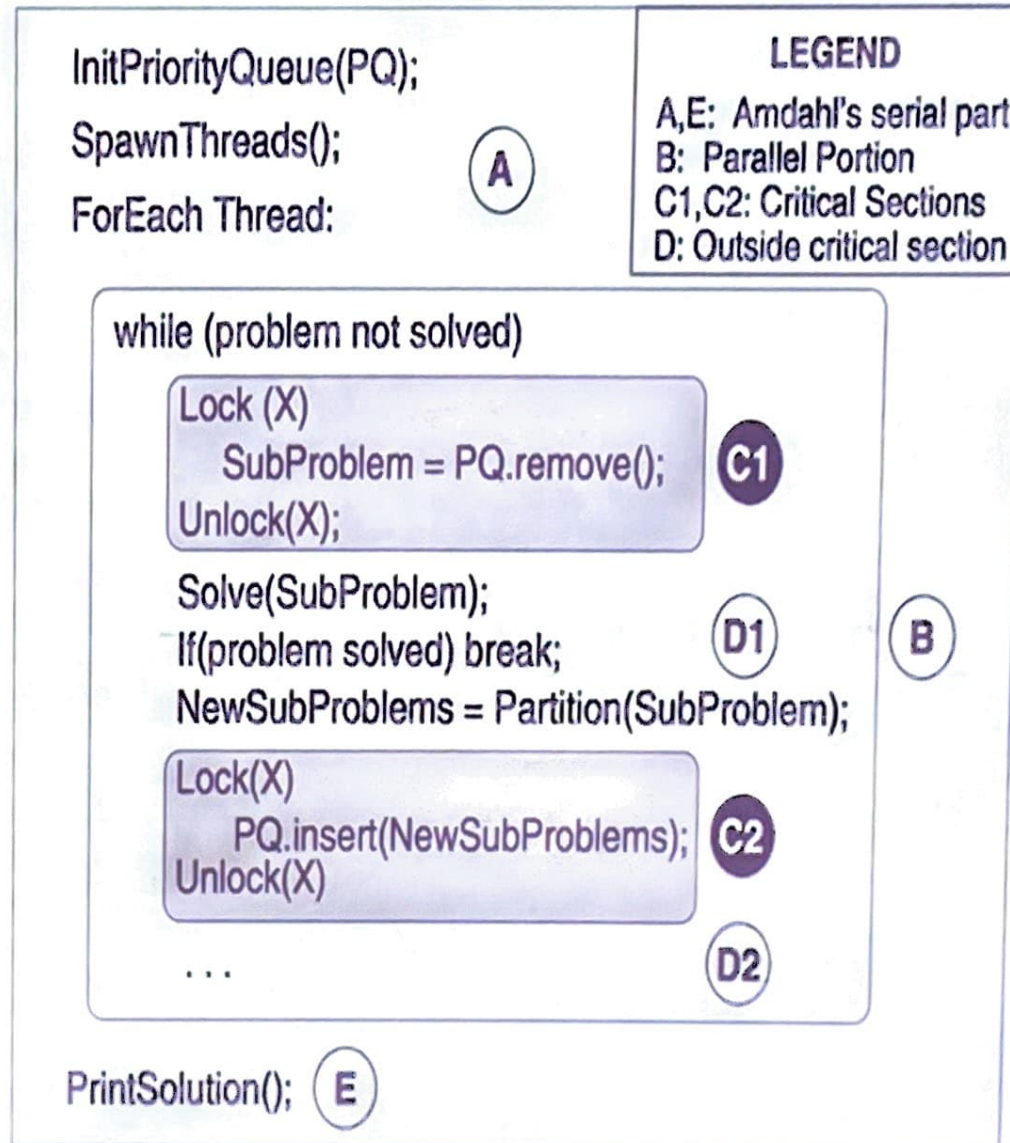
```
for ( i = 0 ; i < N; i++)
```

```
    A[i] = (A[i] + A[i-1]) / 2
```

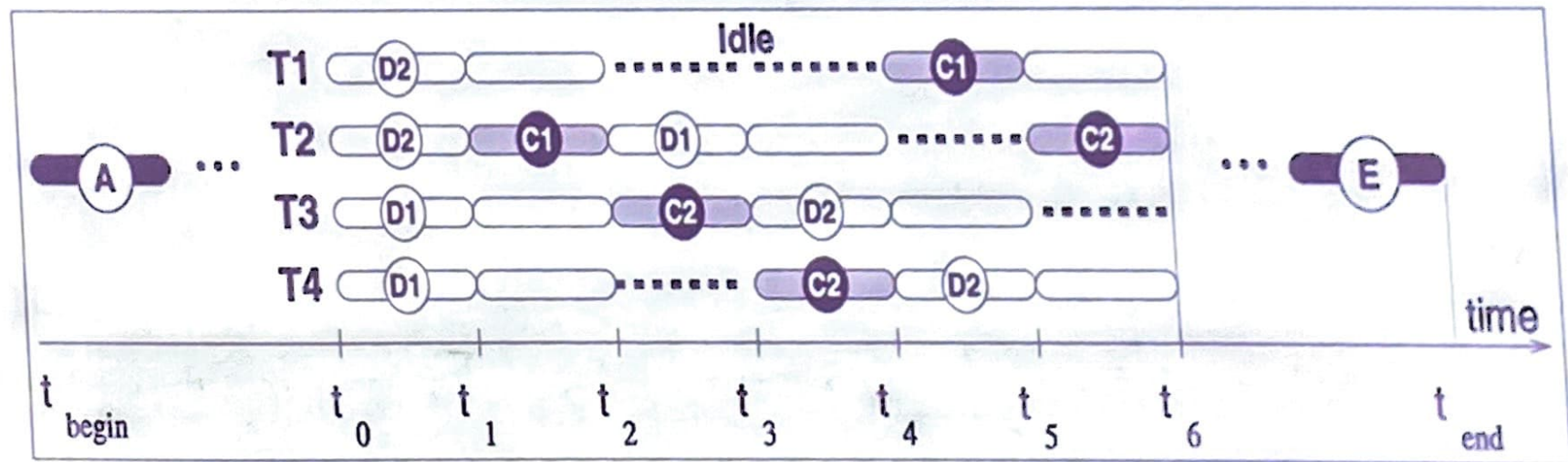
- There are other causes as well:
 - Single thread prepares data and spawns parallel tasks (usually sequential)



Another Example of Sequential Bottleneck (I)



Another Example of Sequential Bottleneck (II)



Bottlenecks in Parallel Portion

- **Synchronization:** Operations manipulating shared data cannot be parallelized
 - Locks, mutual exclusion, barrier synchronization
 - **Communication:** Tasks may need values from each other
 - Causes thread serialization when shared data is contended
- **Load Imbalance:** Parallel tasks may have different lengths
 - Due to imperfect parallelization or microarchitectural effects
 - Reduces speedup in parallel portion
- **Resource Contention:** Parallel tasks can share hardware resources, delaying each other
 - Replicating all resources (e.g., memory) expensive
 - Additional latency not present when each task runs alone